RAS 585: Machine Learning and Artificial Intelligence
Assignment 1 Report

Hunter Flechsig

Fall 2026

1 Overview

This assignment analyzes the Raisin dataset using Bayesian classification, cross-validation, linear regression, and principal component analysis (PCA). The dataset contains 900 samples from two raisin classes, Besni and Kecimen, with seven measured attributes: Area, MajorAxisLength, MinorAxisLength, Eccentricity, ConvexArea, Extent, and Perimeter. Parts 1 and 2 treat all seven measurements as classification inputs. Part 3 treats Perimeter as the dependent variable and the remaining six measurements as predictors.
Note: Full code, intermediate calculations, plots, and program outputs are provided in the submitted Jupyter notebook and notebook PDF. This report presents the main results and findings.

2 Part 1: Bayesian classification

1. Classes

The dataset contains two classes: **Besni** and **Kecimen**. Each class contains 450 samples, so the dataset is balanced.

2. Class-prior log odds

The class prior is

$$P(C_i) = \frac{N_i}{N}.$$

Since both classes contain 450 of the 900 samples,

$$P(C_B) = P(C_K) = 0.5, \qquad \log\left(\frac{P(C_B)}{P(C_K)}\right) = 0.$$

The discriminant written in terms of the log odds is

$$g(x) = \log\left(\frac{P(C_B \mid x)}{P(C_K \mid x)}\right) = \log\left(\frac{p(x \mid C_B)}{p(x \mid C_K)}\right) + \log\left(\frac{P(C_B)}{P(C_K)}\right).$$

Because the prior log odds are zero, classification depends entirely on the feature likelihoods.

3. Class means and covariance matrices

Assuming a separate multivariate-normal distribution for each class, the estimated mean vectors are

$$\mu_B \approx \begin{bmatrix} 112194.79 & 509.00 & 279.62 & 0.8211 & 116675.82 & 0.6912 & 1348.13 \end{bmatrix}^T,$$

$$\mu_K \approx \begin{bmatrix} 63413.47 & 352.86 & 229.35 & 0.7420 & 65696.36 & 0.7078 & 983.69 \end{bmatrix}^T.$$

The sample covariance matrices were estimated separately for Besni and Kecimen using the unbiased sample covariance. The covariance estimates show large differences in scale across variables and strong covariance among size-related measurements such as Area, ConvexArea, and Perimeter.

4. Samples from the fitted distributions

Twenty samples were generated from each fitted multivariate-normal distribution, giving 40 synthetic samples total. The generated values follow the class-specific mean and covariance structure estimated from the original dataset.

5. Joint distribution of MinorAxisLength and Perimeter

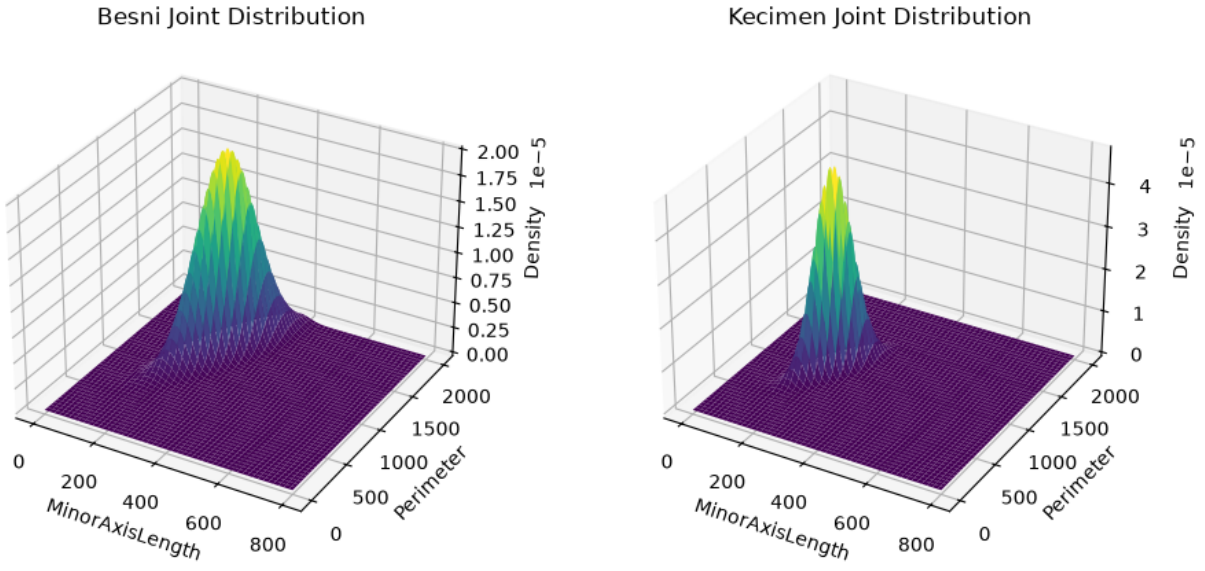
For Besni, the fitted two-variable mean and covariance are

$$\mu_B^{(2)} = \begin{bmatrix} 279.624 & 1348.127 \end{bmatrix}^T, \qquad \Sigma_B^{(2)} = \begin{bmatrix} 2576.949 & 9625.067 \\ 9625.067 & 60912.516 \end{bmatrix}.$$

For Kecimen,

$$\mu_K^{(2)} = \begin{bmatrix} 229.353 & 983.686 \end{bmatrix}^T, \qquad \Sigma_K^{(2)} = \begin{bmatrix} 1159.992 & 3866.030 \\ 3866.030 & 22591.753 \end{bmatrix}.$$

Under the assumed joint multivariate-normal model, each marginal variable is also normally distributed. Therefore, MinorAxisLength and Perimeter are each univariate normal under this model assumption.



(a) Besni fitted joint density.

(b) Kecimen fitted joint density.

Figure 1: Parametric joint distributions of MinorAxisLength and Perimeter.

6. Likelihood ratio

For class C_i with mean μ_i and covariance Σ_i,

$$p(x \mid C_i) = \frac{1}{(2\pi)^{d/2}|\Sigma_i|^{1/2}} \exp\left[-\frac{1}{2}(x-\mu_i)^T\Sigma_i^{-1}(x-\mu_i)\right].$$

The likelihood ratio between Besni and Kecimen is

$$\Lambda(x) = \frac{p(x \mid C_B)}{p(x \mid C_K)} = \frac{|\Sigma_K|^{1/2}}{|\Sigma_B|^{1/2}} \exp\left[-\frac{1}{2}(x-\mu_B)^T\Sigma_B^{-1}(x-\mu_B) + \frac{1}{2}(x-\mu_K)^T\Sigma_K^{-1}(x-\mu_K)\right].$$

7. Separate-covariance discriminant

The class discriminant is

$$g_i(x) = -\frac{1}{2}\log|\Sigma_i| - \frac{1}{2}(x-\mu_i)^T\Sigma_i^{-1}(x-\mu_i) + \log P(C_i),$$

where the common $-\frac{d}{2}\log(2\pi)$ term is omitted. A sample is assigned to the class with the larger discriminant value.

8. Pooled-covariance discriminant

For the pooled model, a shared covariance is used:

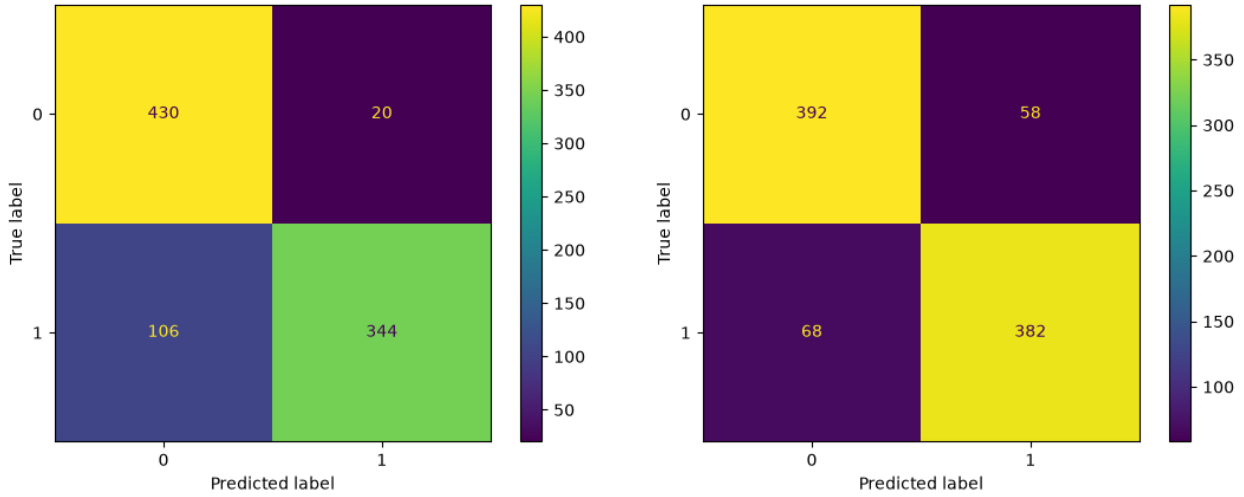
$$\Sigma = \sum_i P(C_i)\Sigma_i.$$

Since the determinant term is common to both classes, the discriminant can be written as

$$g_i(x) = -\frac{1}{2}(x-\mu_i)^T\Sigma^{-1}(x-\mu_i) + \log P(C_i).$$

9. Full-dataset classification results

Both methods achieved an accuracy of **0.860**. The separate-covariance model correctly classified more Kecimen samples, while the pooled model correctly classified more Besni samples. Thus, the covariance assumption changed the distribution of errors even though the overall accuracies were equal.



(a) Separate covariance.

(b) Pooled covariance.

Figure 2: Part 1 confusion matrices. The implementation orders labels as Kecimen followed by Besni.

3 Part 2: Five-fold cross-validation

Five-fold cross-validation was performed with shuffling and a fixed random seed of 42. For every fold, the class means, class covariances, and priors were estimated using only the training portion of that fold. The separate and pooled discriminants were then evaluated on the held-out testing portion. Besni was treated as the positive class when calculating binary classification rates.

1–3. Fold-wise model fitting and classification

The separate-covariance model used one covariance matrix for each class, while the pooled model used a single prior-weighted covariance matrix. Both models were refit independently within each fold so that test samples were not used to estimate the class parameters.

4. Average classification rates

The average results across the five folds were:

Table 1: Five-fold cross-validation results with Besni as the positive class.

Method	TPR	FNR	TNR	FPR	Accuracy
Separate covariance	0.756	0.244	0.947	0.053	0.850
Pooled covariance	0.848	0.152	0.870	0.130	0.858

5. Performance comparison

The pooled model had slightly higher average accuracy, 0.858 compared with 0.850. It also had a higher Besni true-positive rate, while the separate-covariance model had a higher Kecimen true-negative rate.

4 Part 3: Regression and principal component analysis

In Part 3, Perimeter is the dependent variable. The independent variables are Area, MajorAxisLength, MinorAxisLength, Eccentricity, ConvexArea, and Extent. The first 700 observations are used for training and the remaining 200 for testing.

1. Perimeter versus the independent variables

Area, MajorAxisLength, MinorAxisLength, and ConvexArea show clear positive relationships with Perimeter. Eccentricity has a weaker relationship, while Extent shows little clear direct relationship with Perimeter.

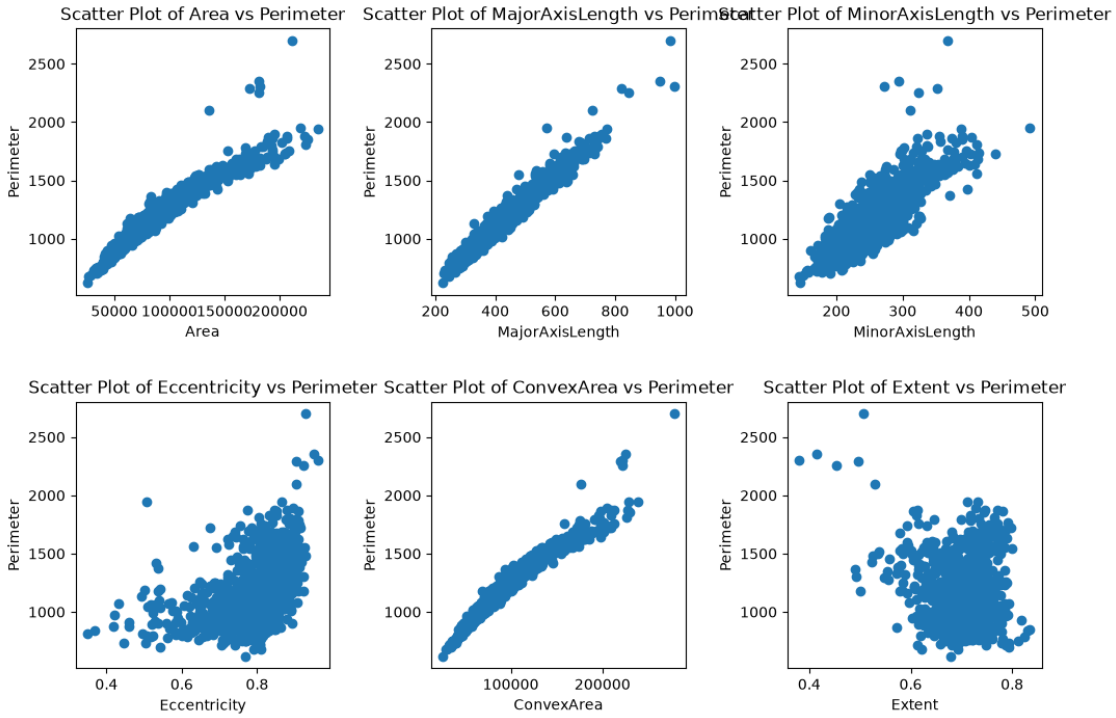


Figure 3: Perimeter plotted against each independent variable.

2. Correlation analysis

The correlation matrix was calculated using only the first 700 training samples. Area, MajorAxisLength, MinorAxisLength, and ConvexArea have the strongest associations with Perimeter. These correlations do not imply causation because correlation measures association and does not establish a causal mechanism.

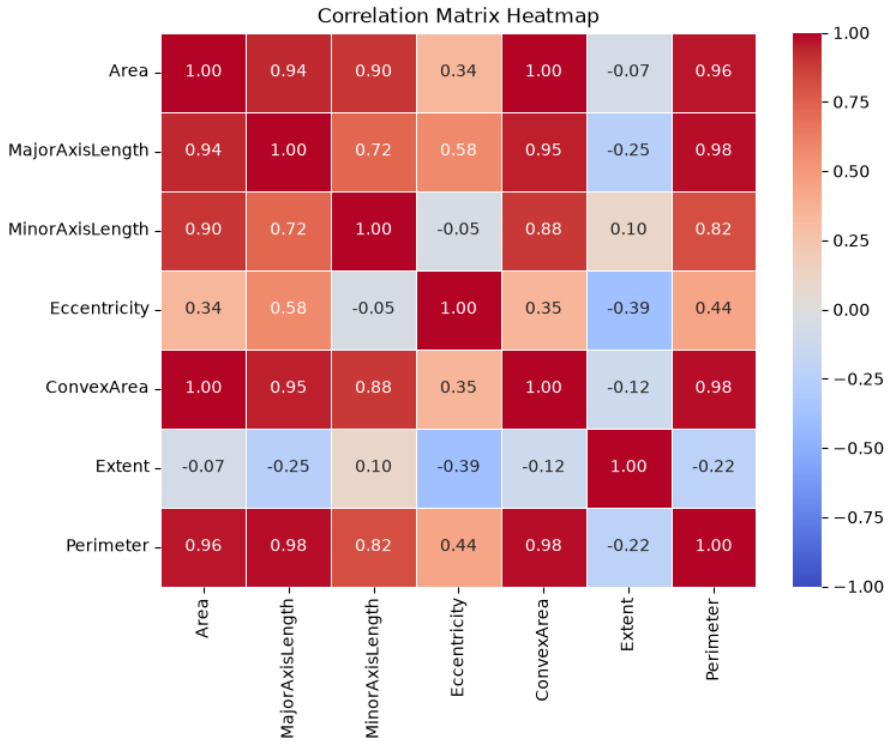


Figure 4: Training-data correlation matrix.

3. Multiple linear regression

A linear regression model was fit using the six independent variables. The fitted raw coefficients from the handwritten implementation were

Area	−0.00774
MajorAxisLength	1.80315
MinorAxisLength	1.60483
Eccentricity	−0.00112
ConvexArea	0.00732
Extent	−0.00067

MajorAxisLength and MinorAxisLength have the largest raw coefficients, but the predictors are measured on very different scales. Therefore, raw coefficient magnitude alone should not be used to determine which variable has the greatest effect.

4. Test-set prediction

On the final 200 samples, the handwritten regression implementation obtained

$$\text{MSE} = 748.97, \qquad R^2 = 0.987.$$

The predicted values closely followed the observed Perimeter values, indicating strong predictive performance despite some larger individual errors.

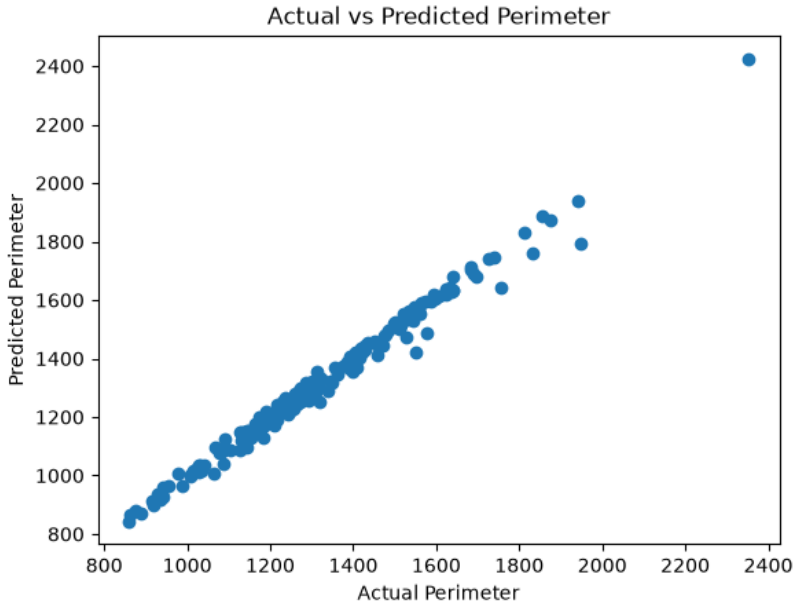


Figure 5: Actual versus predicted Perimeter for the full-feature regression model.

5. Linear dependence among predictors

The predictors are not linearly independent. Area, ConvexArea, MajorAxisLength, and MinorAxisLength have high VIF values, which indicates strong multicollinearity. This can make individual regression coefficients unstable even when prediction accuracy remains high.

6. PCA

Before PCA, the predictors were standardized using parameters fit only on the training data. The fitted PCA explained the following proportions of variance:

Table 2: Variance explained by each principal component.

Component	Variance Explained	Cumulative Variance
PC1	0.6431	0.6431
PC2	0.2417	0.8848
PC3	0.1033	0.9882
PC4	0.0090	0.9972
PC5	0.0023	0.9995
PC6	0.0005	1.0000

The first two components explain about 88.5% of the variance, while the first four explain approximately 99.72%.

7. PCA Pareto chart

The Pareto chart shows that most of the predictor variance is concentrated in the first few components, with very little additional variance contributed by PC5 and PC6.

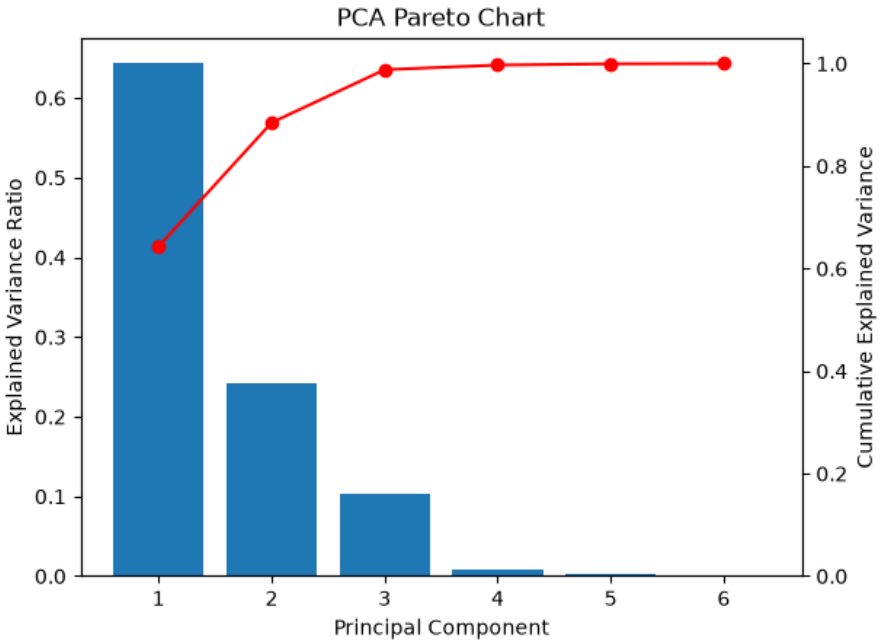


Figure 6: Individual and cumulative explained variance from PCA.

8. Regression using four principal components

A regression model using the first four principal components produced

$$\text{MSE} = 1866.30, \qquad R^2 = 0.967.$$

Although four PCs retain approximately 99.72% of the predictor variance, the prediction error is higher than for the regression using the original variables. This demonstrates that directions explaining predictor variance are not necessarily the directions most useful for predicting Perimeter.

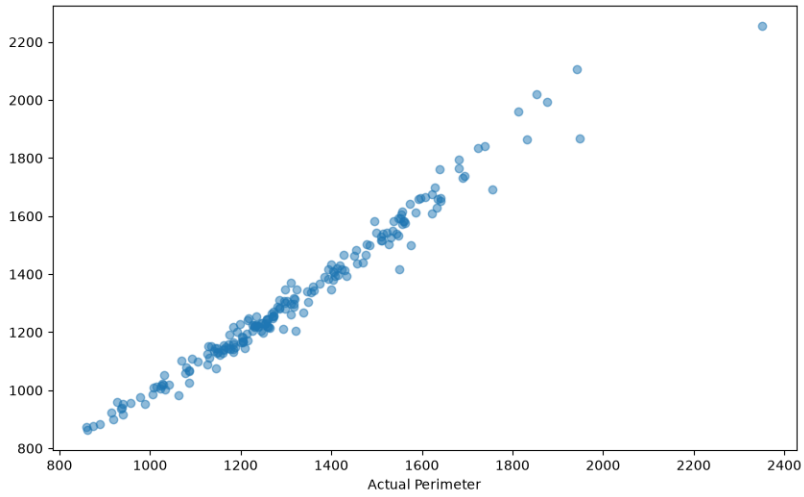


Figure 7: Actual versus predicted Perimeter using the first four principal components.

9. When PCA is useful

A practical use of PCA would be regression on a high-dimensional dataset with many correlated or redundant features, where multicollinearity could make the regression unstable. It can reduce dimensionality and multicollinearity. PCA is less appropriate when the original predictors have important individual meanings and the goal is to interpret their effects, because each principal component is a mixture of the original variables.

5 Comparison of handwritten and AI-generated code

The two notebooks use the same statistical methods but organize them differently. My version uses a two-class `Classifier` built for Besni and Kecimen. The AI version uses NumPy arrays and general helper functions that support any number of classes.

For classification, both versions calculate class means and covariance matrices and use the textbook discriminant equations. My classifier switches between separate and pooled covariance with a class option. The AI version uses separate functions for the two cases. The AI code also specifies the positive class directly when calculating classification rates, while my code controls this through the confusion-matrix label order.

For regression, my notebook uses scikit-learn's `LinearRegression`. The AI notebook uses `statsmodels` OLS and reports extra statistics such as VIF and coefficient significance. Both PCA implementations fit the scaler and PCA model on the training data and then transform the test data. Both show that using four principal components increases prediction error compared with using the original predictors.

The regression test MSE differs between the two implementations. My model gives 748.97, while the AI version gives 665.99. This report uses the result from my implementation.